

Deploying a Docker App on AWS EC2

Auto Scaling Group · Application Load Balancer · ECR · CodeDeploy ·
Jenkins

Step-by-step beginner guide — Region: ap-south-1 (Mumbai) — Ubuntu
26.04

Mubin Shaikh · October 2026

Beginner guide: Docker app on EC2 with ALB, Auto Scaling, CodeDeploy, ECR and Jenkins

Do the phases **in order**. Use one region for everything: **Asia Pacific (Mumbai)** `ap-south-1`. Check the region name in the top-right of the AWS console before every phase.

Replace `<ACCOUNT_ID>` with your 12-digit account number (top-right menu).

What you are building:

```
GitHub -> Jenkins (build image) -> ECR (stores image)
      |
      +--> S3 (bundle) -> CodeDeploy -> EC2 instances in an Auto Scaling Group
Users -> Load Balancer (ALB) -> the EC2 instances (port 3000)
```

Phase 0 - Before you start

1. Sign in to the AWS console. (You are on root; it works, but ideally make an IAM admin user later.)
2. Top-right region dropdown -> choose **Asia Pacific (Mumbai)** `ap-south-1`.
3. Billing safety: search **Budgets** -> *Create budget* -> Zero spend / monthly \$10 -> add your email.
4. Remember: ALB, EC2 and (if used) NAT gateways cost money. Do the **Cleanup** phase at the end.

Phase 1 - ECR repository (stores Docker images)

1. Search **ECR** -> *Elastic Container Registry* -> **Create repository**.
2. Visibility: **Private**. Name: `sample-app`. Leave the rest -> **Create**.
3. Copy the URI, e.g. `<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/sample-app`.

Phase 2 - S3 bucket (stores CodeDeploy bundles)

1. Search **S3** -> **Create bucket**.
2. Name: must be globally unique, e.g. `mubin-codedeploy-bundles-<ACCOUNT_ID>`. Region `ap-south-1`.
3. Keep "Block all public access" **ON**. **Create bucket**. Note the name.

Phase 3 - IAM roles (permissions)

Search **IAM** -> **Roles** -> **Create role**. Create these three.

3a. app-ec2-role (for the app servers)

1. Trusted entity type: **AWS service**. Use case: **EC2** -> Next.
2. Tick these policies (use the search box): `AmazonEC2ContainerRegistryReadOnly`, `AmazonS3ReadOnlyAccess`, `AmazonSSMManagedInstanceCore` -> Next.
3. Role name `app-ec2-role` -> **Create role**.

3b. codedeploy-service-role

1. AWS service. Use case dropdown: type **CodeDeploy**, choose the plain **CodeDeploy** (not "for ECS/Lambda") -> Next.
2. `AWSCodeDeployRole` is pre-attached -> Next. Name `codedeploy-service-role` -> **Create role**.

3c. jenkins-ec2-role

1. AWS service -> **EC2** -> Next.
2. Attach `AmazonEC2ContainerRegistryPowerUser` -> Next. Name `jenkins-ec2-role` -> **Create role**.
3. Open the new role -> **Add permissions** -> **Create inline policy** -> **JSON** and paste (replace the bucket name):

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["s3:PutObject", "s3:GetObject", "s3:ListBucket"],
      "Resource": ["arn:aws:s3:::YOUR-BUCKET", "arn:aws:s3:::YOUR-BUCKET/*"] },
    { "Effect": "Allow",
      "Action": ["codedeploy:CreateDeployment", "codedeploy:GetDeployment",
                 "codedeploy:GetDeploymentConfig", "codedeploy:GetApplicationRevision",
                 "codedeploy:RegisterApplicationRevision", "codedeploy:GetApplication",
                 "codedeploy:GetDeploymentGroup"],
      "Resource": "*" }
  ]
}
```

Name it `jenkins-deploy` -> **Create policy**.

Phase 4 - Security groups (firewalls)

Search **EC2** -> left menu **Security Groups** -> **Create security group**. Use the **default VPC**.

Name	Inbound rules
alb-sg	HTTP, port 80, source 0.0.0.0/0 (and HTTPS 443 later)
app-sg	Custom TCP, port 3000 , source = alb-sg (pick it from the list)
jenkins-sg	Custom TCP port 8080 , source My IP ; SSH 22 from My IP (optional)

Leave outbound as default (all traffic).

Phase 5 - Target group and Load Balancer

5a. Target group

1. EC2 -> **Target Groups** -> **Create target group**.
2. Target type **Instances**. Name `sample-app-tg`. Protocol **HTTP**, port **3000**. VPC default.
3. Health checks: path `/health`. -> Next.
4. Do NOT register any targets (the ASG will) -> **Create target group**.

5b. Application Load Balancer

1. EC2 -> **Load Balancers** -> **Create load balancer** -> **Application Load Balancer**.
2. Name `sample-app-alb`. Scheme **Internet-facing**. IP type IPv4.
3. Network mapping: default VPC, tick **at least 2 Availability Zones** (2 subnets).
4. Security group: remove default, select **alb-sg**.
5. Listener HTTP:80 -> forward to `sample-app-tg`.
6. **Create load balancer**. Note its **DNS name** (you will open it in the browser later).

Phase 6 - Launch template (blueprint for the app servers)

1. EC2 -> **Launch Templates** -> **Create launch template**. Name `sample-app-lt`.
2. Tick "Provide guidance... Auto Scaling" if shown.
3. **AMI**: Quick Start -> **Ubuntu** -> choose **Ubuntu Server 26.04 LTS** (64-bit x86). If 26.04 isn't listed, use "Browse more AMIs", or fall back to **24.04 LTS**.
4. Instance type: `t3.micro` (or `t2.micro` if free-tier).
5. Key pair: "Don't include" is fine (we can use Session Manager instead of SSH).
6. Security groups: select **app-sg**.
7. **Advanced details**:
8. IAM instance profile: `app-ec2-role`
9. User data: paste the contents of `user-data.sh` from this folder, after changing `REGION=us-east-1` to `REGION=ap-south-1`.
10. **Create launch template**.

Servers need internet access to install Docker. Default-VPC subnets give public IPs automatically, so this works.

Phase 7 - Auto Scaling Group

1. EC2 -> **Auto Scaling Groups** -> **Create Auto Scaling group**. Name `sample-app-asg`.
2. Launch template: `sample-app-lt` -> Next.
3. Network: default VPC, select **2+ subnets** in different AZs -> Next.
4. Load balancing: **Attach to an existing load balancer** -> **Choose from your load balancer target groups** -> `sample-app-tg`.
5. Health checks: **tick EC2 only for now** (do NOT tick ELB yet). Grace period 300 s. *Why:* before the first deployment there's no app on the servers, so the ALB check fails and ASG would keep killing them. Switch to ELB after Phase 10 works.
6. Group size: desired **2**, min **2**, max **4**. Scaling policy: **Target tracking**, CPU 50%.
7. Next -> Next -> **Create Auto Scaling group**.
8. Wait ~3 minutes. EC2 -> Instances: you should see 2 new instances **Running**.

Phase 8 - CodeDeploy

8a. Application

1. Search **CodeDeploy** -> **Applications** -> **Create application**.
2. Name `sample-app`. Compute platform **EC2/On-premises** -> **Create**.

8b. Deployment group

1. Inside the app -> **Create deployment group**. Name `sample-app-dg`.
2. Service role: `codedeploy-service-role`.
3. Deployment type: **In-place**.
4. Environment configuration: tick **Amazon EC2 Auto Scaling groups** -> choose `sample-app-asg`.
5. Agent configuration: "Never" (our user-data installs it) or "Now and schedule updates".
6. Deployment settings: `CodeDeployDefault.OneAtATime` (safest for 2 servers).
7. Load balancer: tick **Enable load balancing** -> Application Load Balancer -> select `sample-app-tg`.
8. **Create deployment group**.

Phase 9 - Jenkins server

9a. Launch the instance

1. EC2 -> **Launch instance**. Name `jenkins`. AMI **Ubuntu 24.04/26.04**. Type **t3.small** or bigger (t3.micro is too small).
2. Security group: select existing **jenkins-sg**. Storage 20 GiB.
3. Advanced details -> IAM instance profile: **jenkins-ec2-role**.
4. **Launch**. Then connect: select the instance -> **Connect** -> *EC2 Instance Connect* (or Session Manager if enabled).

9b. Install software (paste in the terminal)

```
sudo apt-get update
sudo apt-get install -y docker.io openjdk-21-jre unzip zip git curl fontconfig
sudo systemctl enable --now docker

# AWS CLI v2
curl -s https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip -o /tmp/a.zip
unzip -q /tmp/a.zip -d /tmp && sudo /tmp/aws/install
aws --version
```

Install Jenkins using the **current official apt instructions** (the signing-key URL changes yearly): <https://www.jenkins.io/doc/book/installing/linux/#debianubuntu> - copy the "Long Term Support release" commands.

Then:

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
sudo systemctl status jenkins      # should say active (running)
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

9c. Set up Jenkins in the browser

1. Open `http://<jenkins-public-ip>:8080`. Paste the admin password.
2. **Install suggested plugins.** Create your admin user.
3. Create the job: **New Item** -> name `sample-app` -> **Pipeline** -> OK.
4. Pipeline section: Definition **Pipeline script from SCM** -> **SCM Git**.
5. Repository URL: `https://github.com/mubinfshaikh/sandbox.git` (if the repo is private: Add credentials -> *Username with password*, username = GitHub user, password = a GitHub personal access token with repo scope).
6. Branch: `*/docker-ec2-asg-loadbalancer`
7. Script Path: `docker-ec2-deploy/Jenkinsfile`
8. **Save.**

9d. Edit values in the repo

In `docker-ec2-deploy/Jenkinsfile` set: `REGION = 'ap-south-1'`, `ACCOUNT = '<ACCOUNT_ID>'`, `BUCKET = '<your bucket>'`. Commit and push to GitHub.

Phase 10 - First deployment

1. Jenkins -> `sample-app` -> **Build Now**. Open **Console Output**. Stages: Build -> Push -> Bundle -> Deploy.
2. CodeDeploy console -> Deployments: watch it go to **Succeeded** (it updates one server at a time).
3. EC2 -> Target Groups -> `sample-app-tg` -> Targets: both **healthy**.

4. Open `http://<ALB-DNS-name>/` in a browser. Refresh a few times; the hostname should switch between the two servers.
5. Now switch the ASG health check to ELB: ASG -> Details -> Health checks -> Edit -> tick **ELB**.
6. Optional: Jenkins -> Configure -> Build Triggers -> **GitHub hook trigger** (add a webhook in the GitHub repo: `http://<jenkins-ip>:8080/github-webhook/`) so each push deploys automatically.

Troubleshooting

Problem	What to check
Jenkins: docker: permission denied	<code>sudo usermod -aG docker jenkins</code> then <code>sudo systemctl restart jenkins</code>
Jenkins: Unable to locate credentials	Instance profile <code>jenkins-ec2-role</code> not attached (EC2 -> Actions -> Security -> Modify IAM role)
ECR push denied	Role missing <code>AmazonEC2ContainerRegistryPowerUser</code> , or wrong region/account in Jenkinsfile
CodeDeploy stuck / "agent not found"	On the server: <code>sudo systemctl status codedeploy-agent</code> ; <code>log /var/log/aws/codedeploy-agent/codedeploy-agent.log</code> ; check user-data ran: <code>/var/log/cloud-init-output.log</code>
Agent won't install on Ubuntu 26.04	New OS may be unsupported; use Ubuntu 24.04 AMI, or build the agent from the <code>aws-codedeploy-agent</code> GitHub source
Script failed in hook	<code>/opt/codedeploy-agent/deployment-root/<dep-id>/<group-id>/logs/scripts.log</code>
ECR pull denied on server	<code>app-ec2-role</code> missing ECR read policy
Targets unhealthy	<code>app-sg</code> must allow 3000 from <code>alb-sg</code> ; <code>curl localhost:3000/health</code> on the server; <code>docker ps</code>
ALB URL times out	<code>alb-sg</code> must allow 80 from anywhere; ALB in public subnets

Cleanup (stop charges)

Delete in this order: Auto Scaling group -> Load balancer -> Target group -> CodeDeploy app -> Jenkins instance -> Launch template -> ECR images/repo -> S3 bucket -> (IAM roles and security groups are free).

Appendix: project files

All files live in `docker-ec2-deploy/` of the repository.

Jenkinsfile

```
// Jenkins job "Script Path": docker-ec2-deploy/Jenkinsfile
pipeline {
  agent any
  environment {
    REGION = 'us-east-1'           // change
    ACCOUNT = '123456789012'       // change
    BUCKET = 'myapp-codedeploy-bundles'
    CD_APP = 'sample-app'
    CD_DG = 'sample-app-dg'
    REPO = "${ACCOUNT}.dkr.ecr.${REGION}.amazonaws.com/sample-app"
  }
  stages {
    stage('Build') {
      steps {
        script { env.TAG = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}" }
        dir('docker-ec2-deploy') { sh 'docker build -t $REPO:$TAG -t $REPO:latest .' }
      }
    }
    stage('Push') {
      steps {
        sh '''
          aws ecr get-login-password --region $REGION | docker login --username AWS --p
assword-stdin $ACCOUNT.dkr.ecr.$REGION.amazonaws.com
          docker push $REPO:$TAG
          docker push $REPO:latest
        '''
      }
    }
    stage('Bundle') {
      steps {
        dir('docker-ec2-deploy') {
          sh '''
            echo "IMAGE=$REPO:$TAG" > image.env
            zip -r bundle-$TAG.zip appspec.yml scripts image.env
            aws s3 cp bundle-$TAG.zip s3://$BUCKET/bundle-$TAG.zip --region $REGION
          '''
        }
      }
    }
    stage('Deploy') {
      steps {
        sh '''
          ID=$(aws deploy create-deployment --region $REGION \
            --application-name $CD_APP --deployment-group-name $CD_DG \
            --s3-location bucket=$BUCKET,key=bundle-$TAG.zip,bundleType=zip \
            --query deploymentId --output text)
        '''
      }
    }
  }
}
```

```

        aws deploy wait deployment-successful --deployment-id $ID --region $REGION
    ...
    }
}
}
}
}

```

appspec.yml

```

version: 0.0
os: linux
files:
  - source: /
    destination: /opt/app
hooks:
  ApplicationStop:
    - location: scripts/stop.sh
      timeout: 60
  BeforeInstall:
    - location: scripts/pull.sh
      timeout: 300
  ApplicationStart:
    - location: scripts/start.sh
      timeout: 120
  ValidateService:
    - location: scripts/health.sh
      timeout: 60

```

user-data.sh

```

#!/bin/bash
# EC2 launch template user data (Ubuntu). Set REGION before use.
set -e
REGION=us-east-1
apt-get update
apt-get install -y docker.io ruby-full wget unzip curl
systemctl enable --now docker
usermod -aG docker ubuntu

curl -s https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip -o /tmp/awscli.zip
unzip -q /tmp/awscli.zip -d /tmp && /tmp/aws/install

cd /tmp
wget https://aws-codedeploy-$REGION.s3.$REGION.amazonaws.com/latest/install
chmod +x install && ./install auto
systemctl enable --now codedeploy-agent

```

scripts/pull.sh

```

#!/bin/bash
set -e
# image.env ("IMAGE=<ecr-uri>:<tag>") sits at the root of the deployment archive

```

```
source "$(dirname "$0")/../image.env"
REGISTRY="${IMAGE%%/*}"
REGION=$(echo "$REGISTRY" | cut -d. -f4)
aws ecr get-login-password --region "$REGION" | docker login --username AWS --password-
stdin "$REGISTRY"
docker pull "$IMAGE"
```

scripts/start.sh

```
#!/bin/bash
set -e
source "$(dirname "$0")/../image.env"
docker rm -f app 2>/dev/null || true
docker run -d --name app --restart unless-stopped -p 3000:3000 \
  -e APP_VERSION="${IMAGE##*:}" "$IMAGE"
```

scripts/stop.sh

```
#!/bin/bash
docker stop app 2>/dev/null || true
docker rm app 2>/dev/null || true
exit 0
```

scripts/health.sh

```
#!/bin/bash
for i in $(seq 1 15); do
  curl -fs localhost:3000/health && exit 0
  sleep 2
done
exit 1
```

Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY app/package*.json ./
RUN npm ci --omit=dev
COPY app/ .
EXPOSE 3000
CMD ["node", "server.js"]
```

app/server.js

```
const http = require('http');
const os = require('os');
http.createServer((req, res) => {
  if (req.url === '/health') { res.writeHead(200); return res.end('ok'); }
  res.writeHead(200, {'Content-Type': 'text/plain'});
```

```
res.end(`Hello from ${os.hostname()} (version ${process.env.APP_VERSION || 'dev'})\n`);\n}).listen(3000, () => console.log('listening on 3000'));
```

app/package.json

```
{"name": "sample-app", "version": "1.0.0", "main": "server.js", "scripts": {"start": "node server.js"}}
```